

PRACTICAL 3

Tashil Haripersad (u25466004)

Kyle Naick (u25040040)

Haytham Phillips (u25030907)

Task 1: Event concept and complete architecture

1.1 TechConnect is a two-day technology conference held at a single campus venue, combining keynote talks, hands-on workshops, a sponsor exhibition, and attendee stats under one control centre. Event is large enough to need multiple physical buildings; each divided into zones. Each one containing several independent operating units that should respond differently to the same event. (weather, capacity, power, announcements, etc)

Concrete event unit types

- **LectureHall:** a keynote/talk room; on WEATHER_ALERT it keeps running if indoors but pauses if outdoors; a PAUSE always pauses it, resuming on RESUME; updates its schedule on SCHEDULE_CHANGE, EVACUATE stops the talk immediately; CANCEL/CLOSE closes the hall.
- **WorkshopRoom:** a hands-on workshop; on WEATHER_ALERT only pauses event when outdoors, but a direct PAUSE always pauses it; EVACUATE stops the session immediately, SCHEDULE_CHANGE replans exercises for the new slot rather than just recording new times.
- **RegistrationDesk:** check-in point; opens/closes on OPEN/CLOSE, pauses badge issuing on WEATHER_ALERT (only when outdoors) and on PAUSE, stops immediately on EVACUATE, shuts down on CANCEL
- **ExhibitionBooth:** a sponsor booth; pauses live demos on WEATHER_ALERT (outdoors only) or PAUSE. Treats EVACUATE same as CANCEL: the booth is vacated/packed up immediately rather than being paused
- **CateringStation:** food vendor; POWER_OUTAGE, switches to cold-food fallback menu instead of pausing and automatically restores the hot menu on the next RESUME.
- **ShuttleStop:** transport pickup point; pauses pickups on WEATHER_ALERT and halts on EVACUATE but has no reaction at all to general PAUSE/CAPACITY_ALERT, or CANCEL.
- **MedicalStation:** on-site first aid; remains active through EVACUATE and switches to backup battery for POWER_OUTAGE. Everything else is the default except CANCEL, they stand down.

Grouping/areas (composites)

- **Root (TechConnect Conference):** top-level for the whole event
- **Zones:** functional areas, each containing the individual units
 - **Keynote Zone:** LectureHall × 3, CateringStation
 - **Workshop Zone:** WorkshopRoom × 2
 - **Sponsor Zone:** ExhibitionBooth × 2
- **Leaves:** RegistrationDesk, MedicalStation, CateringStation, ShuttleStop

1.2 Composite

- **Component:** EventComponent
- **Leaf:** RegistrationDesk, LectureHall, WorkshopRoom, ExhibitionBooth, MedicalStation, CateringStation, ShuttleStop
- **Composite:** EventGroup
- **Client:** main()

Observer

- **Subject:** Subject
- **ConcreteSubject:** EventControl, EventGroup
- **Observer:** Observer
- **ConcreteObserver:** EventGroup, RegistrationDesk, LectureHall, WorkshopRoom, ExhibitionBooth, MedicalStation, CateringStation, ShuttleStop

Classes participating in more than one role

- EventGroup has three separate roles:
 - **Composite:** it owns and recursively delegates to its children (open (), close (), reportStatus (), getCapacity ()).
 - **Observer:** it is attached to its parent so it hears notices cascading down from above.
 - **Subject:** it is simultaneously a subject to its own children, cascading whatever notice it receives further down the tree. This allows a single notice travel through several runtime levels.
- RegistrationDesk, LectureHall, WorkshopRoom, ExhibitionBooth, MedicalStation, CateringStation, ShuttleStop participate in two roles:
 - **Leaf:** it is atomic and has no children
 - **Observer:** it reacts to notices pushed from whatever Subject it is currently attached

1.4 (a) It's not just a nested type hierarchy; it models real containment. Beyond the Algorithm (a LectureHall) isn't merely associated with Keynote Zone, it's physically inside it, which is physically inside TechConnect Conference. Whole-level operations are defined recursively in terms of parts: getCapacity() sums every child, open()/close() propagate to every child, reportStatus() prints the whole subtree; so root->getCapacity() genuinely means "everything the conference contains," not a count of top-level items. Most importantly, membership is enforced by ownership: add()/removeChild() guarantee every EventComponent belongs to exactly one EventGroup at a time, so moving a part (e.g. relocating the CloudNova booth) is an explicit, atomic transfer of membership, not just adding a second reference. That ownership discipline is what separates a true part-whole tree from an ordinary graph of associations.

(b) EventControl and EventGroup can't know at compile time which concrete classes need to hear about a Notice, because that set changes at runtime; First Aid gets attached directly to EventControl mid-simulation and later detached without touching EventControl or its zone; the booth relocation changes which EventGroup cascades notices to CloudNova without modifying either group's class; and seven leaf types react differently to the same Notice, which the rules forbid achieving via type-checks or switch-on-concrete-type. If EventControl instead hard-coded calls like root->pauseHall(), it would have to know every leaf type and be edited every time one was added. Observer inverts that dependency: Subject::notify() only ever calls the abstract Observer::update(), so EventControl and EventGroup depend on nothing more specific than that interface; any object implementing it, now or later, can register without changing Subject or Composite code.

(c) Entirely EventGroup. Each EventGroup owns its children through a vector of raw, owning pointers (no smart pointers per project constraints). add() takes ownership of a new or transferred child; removeChild() releases ownership to the caller and removes it from the list; the destructor deletes everything still owned. Because a component is owned by at most one EventGroup at a time, this can never double-delete or leak. The root EventGroup itself is owned by main(), deleted explicitly at shutdown, cascading through the whole subtree exactly once. EventControl owns no EventComponents at all; it sits outside the Composite tree entirely and only holds non-owning Observer* references.

(d) Nobody, in the ownership sense; Subject always stores non-owning pointers, and never calls new/delete on an Observer. An Observer's real lifetime is governed by whatever structurally owns it (usually Composite ownership). Because Subject doesn't own observers, destroying one never deletes the other. The responsibility this places on the true owner is handled explicitly: removeChild() detaches an outgoing child from its old parent's observer list in the same call that releases ownership, and main.cpp explicitly detaches root from EventControl before deleting it. Because ownership and observation are tracked independently, the same Observer can register with more than one Subject at once; demonstrated when First Aid is attached directly to EventControl while still owned by, and registered with, the root group.

(e) EventGroup is deliberately both. As Composite, it answers "what's physically inside me?"; an ownership relationship to its children. As Observer, it answers "who do I need to hear from?"; a relationship to its parent. As Subject, it answers "who needs to hear from me?"; a relationship to its children again, but through a completely separate mechanism (its own observer list, not its ownership list). These relationships are provably separable, not just conceptually: EventControl attaches directly to First Aid, a unit it doesn't own; and EventGroup::add() only auto-registers a child as observer because it happens to also implement Observer; a future component that didn't wouldn't be forced into that coupling. Finally, this dual role is exactly what makes cascading work: a Notice from EventControl reaches leaves three levels down only because each intermediate EventGroup can simultaneously receive (Observer) and retransmit (Subject) it. Removing either role would break the cascade, not simplify the design; proof that both roles are doing real, distinct work.

Task 2: Composite event structure

2.4 When delete root; is called it invokes EventGroup::~~EventGroup() on the root instance. observers_.clear() ensures that no new updates/notifications can be sent. The destructor iterates over the children_ vector deleting each child. When each child is deleted the correct destructor is called polymorphically. If the child has its own observer list (vector) then it will call the destructor recursively on all of its children. If the child is a leaf the destructor completely deallocates that leaf. In this manner the "tree" is deleted bottom up.

The single release policy ensures every object created, is freed exactly once. Every component (e.g. hall, booth, etc) is stored in the children_ vector of precisely one parent. No two branches share the same pointer to a children vector. Calling delete root; loops through each children_ vector and calls delete on each child. Each vector does the same for each object in the vector. This ensures that each object is visited once and deleted once.

When an object is transferred between composites at runtime for example moving boothA to from sponsorZone to exhibitionWing, ownership must be managed with a strict multi-step protocol to prevent memory leaks. An EventGroup calls `removeChild()` which finds the component and erases its `children_` vector and removes it from its `observers_` list. `removeChild()` does not deallocate the memory for the object. It releases ownership and returns a pointer to the context. During this phase, the pointer is forwarded into the destination composite with `newParent->add()`. Now ownership and observer registrations are re-established under the new parent. If the pointer is not reattached then it must be deleted. After `add()` is called the pointer is now stored within the parents' own `children_` vector and re-registers the object as an observer. This ensures that object is never owned by more than one EventGroup at one time. This guarantees that when `delete root;` is called the transferred component is deleted exactly once.

Task 3: Observer notification system

3.5 EventFlow uses a push model: `Subject::notify()` calls `obs->update(notice)` on every registered observer, sending a full Notice object instead of just a signal. The Notice struct in Notice.h has three fields, NoticeType (e.g. CAPACITY_ALERT), a message string for logging, and an optional threshold, that's everything any observer will get. No observer ever calls back to the Subject for more data. The trade-off is between coupling and payload size, since they only implement `update()` and never need a pointer back to query the subject, hence there are no stale reads. Also, the whole notice arrives at once instead of being fetched later after a state could have changed. This suits how our EventGroup works since it both reacts to a notice and forwards the same one to its children with no extra query steps. The cost is that every observer receives all three fields even when it only needs one, but with only a few notice types in our design, that small waste is a fair price for not having to deal with coupling and staleness issues.

Task 4: Event rules and dynamic behaviour

4.1 Five event rules causing concrete units to react differently to the same notice

- **CAPACITY_ALERT**, same notice, different outcome from identical objects. Beyond the Algorithm (a LectureHall) is set to 295/280 attendance and logs "VENUE FULL"; Quantum Leaps (also a LectureHall, same class) is set to 150/280 and logs "not yet full". The difference comes entirely from each object's own `currentAttendance_`/capacityThreshold_ state, not from type.
- **WEATHER_ALERT** affects outdoor units only, regardless of leaf type. Bridging the Divide (outdoor LectureHall) and Coffee Station (outdoor CateringStation) react; Beyond the Algorithm (indoor LectureHall) and Radisson Blu Catering (indoor CateringStation) do not; same class, different reaction, driven by the `indoors_` flag.
- **POWER_OUTAGE** produces two distinct fallback behaviours. CateringStation switches to a cold-food menu; MedicalStation switches to backup battery power; neither pauses or closes, unlike every other notice-driven state change in the system.
- MedicalStation never truly "stops". Across **PAUSE/RESUME/CANCEL/WEATHER_ALERT** (all of which it's excluded from reacting to) it simply has no case, while EVACUATE produces "remaining active, assisting evacuation" rather than closing; the one leaf type designed to stay operational through almost everything.

- **PAUSE/RESUME/SCHEDULE_CHANGE/CANCEL** only affect programme-facing leaves. LectureHall, WorkshopRoom, RegistrationDesk, ExhibitionBooth react; ShuttleStop (transport must keep running) and CateringStation/MedicalStation (unrelated concerns) don't; achieved purely by which classes have a case, never a controller checking typeid/dynamic_cast against concrete types.

4.4 Three original features and why none creates a god object

- **POWER_OUTAGE**: touches only CateringStation/MedicalStation; every other class simply never gained a case, so EventControl and EventGroup needed zero changes.
- **CANCEL**: touches only the four programme-facing leaves, reusing their existing isOpen_ state; no new fields, no shared interface.
- **ANNOUNCEMENT**: the inverse: touches only EventGroup (one case, one class); no leaf type gained any awareness it exists.

Task 7: Git and GitHub engineering workflow

<https://github.com/Kyle-Naicker/EventFlow>

7.4 Our team divided work along the two GoF patterns at the core of EventFlow, plus integration.

We created three long-lived branches beyond main: composite, observer, and dev. Haytham worked primarily on composite, implementing EventComponent, EventGroup, and the concrete EventUnit leaf types. Kyle worked primarily on observer, implementing Subject, Observer, EventControl, and the Notice representation. Tashil worked on dev, which served as the integration branch where main.cpp was built once both collaborations were in place, and where the two branches were combined and tested together as a working simulation.

Changes were integrated through pull requests rather than direct merges to main. Once composite and observer were independently functional, each was opened as a pull request into dev, where the team reviewed the difference together, resolved a small number of merge conflicts around EventComponent.h (since both branches needed to reference it), and confirmed the combined build compiled and ran cleanly with make. Only after dev had a fully working, tested simulation was a final pull request opened from dev into main. This gave us a working integration branch to test against before anything touched main, so main only ever received code the whole team had already seen run successfully.

Two points in our history show this collaboration concretely:

- The pull request merging composite into dev, which the team reviewed together and where we flagged that EventGroup::add() needed to auto-attach children as observers, not just take ownership, a design point later documented in our rationale.
- The pull request merging observer into dev, which surfaced a merge conflict in EventComponent.h between the two branches' independent additions, resolved jointly rather than by whoever pushed first.

We also used pair programming for the trickiest parts of the design. Particularly the Composite/Observer coupling inside EventGroup::add(), and the runtime reorganisation logic in removeChild(), following the Driver/Navigator/Tester rotation from the COS 214 pair

programming guidelines, rotating roles roughly every 20–30 minutes so each member both wrote and reviewed the ownership/observer transfer logic before it was merged.